Практическое задание 2. Циклы

Команды для Linux:

CTRL+C – завершить программу.

```
# Команды в терминале
mkdir my_dir # Создать каталог my_dir
ls           # Просмотреть список файлов в текущем каталоге
pwd          # Вывести имя текущей директории
cd my_dir    # Перейти в каталог my_dir
cd ..        # Подняться на каталог "выше" (вернуться назад)
nano hello.c # открыть файл в текст. редакторе nano
rm file_name # удалить файл с именем file_name
man command  # Мануал (Описание команды)

# Команды в текст. редакторе nano
CTRL+O (^O) # save program
CTRL+X (^X) # exit program

# Компиляция и запуск
gcc hello.c -o hello
./hello
```

На **оценку 3** = 1 алгоритм, на **оценку 4** = 3 алгоритма, на **оценку 5** = 5 алгоритмов.

Реализовать алгоритмы начиная с вашего варианта: вариант берем из списка группы. Всего алгоритмов 17. Если я в списке 7 и хочу оценку 5, то я возьму алгоритмы 7, 8, 9, 10, 11. Если я в списке 23 и тоже хочу оценку 5, то начну с варианта, который получается если взять остаток от деления на общее число доступных алгоритмов: 23 % 17 = 6, следовательно буду выполнять 6, 7, 8, 9, 10 алгоритмы. Если я 15, то на оценку 5 мне надо: 15, 16, 17, 1, 2.

Для реализации алгоритмов необходимо сам алгоритм обернуть в функцию main() и не забыть include. Скомпилировать, исправить все ошибки, скомпилировать снова и запустить.

Определить, что вычисляет фрагмент (Вычисляет простое ли число, НОД, факториал и т.п.) и вывести на экран результат.

Сдаем лабораторную, когда выполнены все N алгоритмов на ожидаемую оценку. Алгоритмы можно реализовать в одном .c файле либо под каждый алгоритм отдельный .c файл

```
//---------------------------------------------------- 1
for (i=n1; !(n1 % i ==0 && n2 % i ==0); i--)
    printf("n1 n2\n");
//---------------------------------------------------- 2

for (n=a; n%a!=0 || n%b!=0; n++)
    printf("n a b\n");
//---------------------------------------------------- 3
for (n=2; n<a; n++)
     if (a%n==0) break;
if (n==a) puts("1");
else puts("0");
//---------------------------------------------------- 4
for (n=2; n<a && a%n!=0; n++);
if (n==a) puts("1");
else puts("0");
//---------------------------------------------------- 5
for (s=0,n=2; n<a; n++)
     if (a%n==0) s++;
if (s==0) puts("1");
else puts("0");
//---------------------------------------------------- 6
for (s=0,n=2; n<a; n++)
     if (a%n==0) { s=1; break; }
//---------------------------------------------------- 7
n=2, flag = 0;
while(a%n!=0)
{
     n++;
     if (n==a) {flag = 1; break};
}
//---------------------------------------------------- 8
for (s=1, i=1; i<=n; i++)
    s = s * i;
//---------------------------------------------------- 9
for (n=a, s=0; n!=0; n=n/10)
     { k=n%10; if (k>s) s=k;}
//---------------------------------------------------- 10
for (n=a, s=0; n!=0; n=n/10)
     { k=n%10; s=s*10+k;}
```

```
//------------------------------------------------------- 11
for (k=0, m=1; m <= n; k++, m = m * 2)
      { printf("%d", k-1); }
//-------------------------------------------------------12
for (n=a, s=0; n!=0; n=n/10)
      { k=n%10; s=s+k; }
//-------------------------------------------------------13
for (s=1, i=0; i<=n; i++) s = s * 2;
//-------------------------------------------------------14
for (n=2; n<a; n++)
     { if (a%n==0) break; }
if (n==a) puts("Good");
//-------------------------------------------------------15
for (s=0,n=2; n<a; n++)
     { if (a%n==0) s++; }
if (s==0) puts("Good");
//-------------------------------------------------------16
for (i=2; i<a; i++)
     if (a%i==0) break;
if (i==a) puts("Good");
//------------------------------------------------------- 17
n=2, flag = 0;
while(b%n!=0){
   n++;
   if (n==b) {
      flag = 1;
      break;
   }
}
```

Под Звездочкой (+ балл):

Асимметричный алгоритм RSA - это фундамент безопасности современного интернета: от HTTPS и SSH-ключей до электронных подписей. Его надёжность держится на простой, но мощной математической идее: перемножить два больших простых числа для процессора - секундное дело, а вот разложить результат обратно на множители (факторизовать) практически невозможно. Именно поэтому открытый ключ можно спокойно публиковать, а секретный - хранить в тайне.

Однако администратор системы Боб поленился и выбрал слишком маленькие **p** и **q**, сломав всю магию криптографии. Необходимо побыть в роли хацкера и методом перебора (brute-force) взломать перехваченное сообщение, которое Боб отправил Алисе, вычислив секретный ключ **d** и восстановив скрытый за ним символ.

Алгоритм атаки

1. **Факторизация составного модуля:** найти два исходных **простых** числа p и q перебором меньших делителей n, зная, что их произведение $n = p * q$
2. **Расчет функции Эйлера:** вычислить скрытый параметр системы по формуле $\varphi = (p - 1) * (q - 1)$
3. **Перехват секретного ключа:** найти секретную экспоненту d, для которой будет истинно условие: $(d * e) \% \varphi == 1$, где e - открытый ключ
4. **Финальное чтение сообщения:** восстановить ASCII-код символа $symb = (cipher^d) \% n$

Подсказка: не забывайте про переполнения (Overflow) при математике с целыми числами ($cipher^d$ может быть большим) – используйте модульное возведение в степень в цикле: после каждого умножения берем остаток по модулю n.

Перехваченные данные:

Открытый ключ (e)	Модуль (n)	Сообщение
17	35026003	4670998
17	45222511	3357214
17	55293143	49786238
17	62832893	43592627
19	25273609	8387449